

Procesamiento del Lenguaje Natural I

EXPRESIONES REGULARES

Soto Montalvo Herranz

Objetivos

- Introducir el concepto de expresión regular.
- Ver los usos de las expresiones regulares en tareas del procesamiento del lenguaje natural.

Índice

- Introducción
- Expresiones regulares en PLN
- Tipos de expresiones regulares
- Expresiones regulares en Python

Introducción

- Una expresión regular es la notación estándar para la caracterización de una secuencia de caracteres (letras, números, espacios, tabuladores y signos de puntuación).
- La secuencia de caracteres forma un patrón, que ayuda a encontrar las coincidencias en los textos.
- Las expresiones regulares permiten buscar, validar y manipular texto de manera eficiente.
- También se les denomina regex o regexp.

Expresiones regulares en PLN

- En tareas de Procesamiento del Lenguaje Natural es frecuente la utilización de expresiones regulares.
 - Las ER juegan un papel fundamental en PLN porque permiten buscar texto de forma eficiente.
- Ejemplos de uso:
 - Diferentes análisis del lenguaje
 - Recuperación de información
 - Detección de Entidades Nombradas
 - Extracción de Información
 - Etc.

Tipos de expresiones regulares

- Coincidencia literal
- Clausura de Kleene
- Grupos y alternativas
- Cuantificadores
- Caracteres especiales

Tipos de expresiones regulares

- Coincidencia literal o expresión regular básica
 - Es la ER más simple, donde una secuencia de caracteres tiene que coincidir exactamente con la cadena de texto.
 - Ejemplo: “mundo”, tendrá que coincidir exactamente con “mundo”
- Las ER son case-sensitive, se distingue entre mayúsculas y minúsculas

Tipos de expresiones regulares

- Clausura de Kleene
 - Operador *: cero o más ocurrencias de la ER inmediatamente anterior

$ba^* \rightarrow b, ba, baa, baaa, \dots$

- Operador +: una o más ocurrencias de la ER inmediatamente anterior

$ba^+ \rightarrow ba, baa, baaa, \dots$

Tipos de expresiones regulares

- Grupos y alternativas
 - El símbolo | se utiliza para representar una alternativa.

$a|b \rightarrow a \text{ o } b$

- Los paréntesis () se utilizan para agrupar partes de la expresión regular.

$(ba)^+ \rightarrow ba, baba, bababa, \dots$

$(b|a)^+ \rightarrow b, a, bb, ba, aa, ab, \dots$

Tipos de expresiones regulares

- Cuantificadores. Definen cuántas veces debe aparecer un carácter o un grupo en la cadena. Algunos ejemplos son:

- $\{n, m\}$: de n a m repeticiones de la ER que lo precede

$(ba)\{2,3\} \rightarrow baba, bababa$

- $\{n\}$: coincide con exactamente n repeticiones.

$(ba)\{3\} \rightarrow bababa$

- $\{n, \}$: repite al menos n veces o más.

Tipos de expresiones regulares

- Caracteres especiales. Son metacaracteres que tienen significados especiales en las expresiones regulares.
 - . (punto): Este símbolo coincide con cualquier carácter excepto el salto de línea.
beg.n → begin, begun, begxn, ...
a.*b → cualquier cadena que empiece con a y termine con b
 - ? (signo de interrogación): coincide con cero o una repetición de lo anterior.
(a|b)?c → ac, bc, c
colo?r → colr o color

Tipos de expresiones regulares

- Caracteres especiales. Son metacaracteres que tienen significados especiales en las expresiones regulares.

- [] (corchetes): agrupa caracteres en grupos o clases.

[a-z] → especifica el rango de caracteres que va de la a a la z.

- \ (barra invertida): Escapa un carácter especial para tratarlo como literal.
 - \$ Signo de dólar. Coincide con el final de la cadena.

Tipos de expresiones regulares

- Caracteres especiales. Son metacaracteres que tienen significados especiales en las expresiones regulares.
 - ^ Signo de intercalación. Coincide sólo al principio de la cadena

Si se usa en combinación con los corchetes sirve para indicar que algo no esté presente.

If the caret ^ is the first symbol after the open square brace [, the resulting pattern is negated

`[^A-Z]` Not an upper case letter

`[^a-z]` Not a lower case letter

`[^Ss]` Neither 'S' nor 's'

Tipos de expresiones regulares

- Caracteres especiales.

\d	— Representa un dígito del 0 al 9.
\w	— Representa cualquier carácter alfanumérico.
\s	— Representa un espacio en blanco.
\D	— Representa cualquier carácter que no sea un dígito del 0 al 9.
\W	— Representa cualquier carácter no alfanumérico.
\S	— Representa cualquier carácter que no sea un espacio en blanco.

- Hay muchos más. Consultar la [biblioteca estándar](#).

Tipos de expresiones regulares

- Precedencia de operadores, ordenados de mayor a menor precedencia:

Parenthesis ()

Counters * + ? {}

Sequences and anchors ^ \$

Disjunction |

Expresiones regulares en Python

- El módulo `re` en Python es el que da soporte para las expresiones regulares.
- Algunos de los métodos que tiene:

`search()`: searches for a match anywhere in the string

`findall()`: returns a list of all non-overlapping matches in the string

`finditer()`: returns an iterator yielding match objects

`sub()`: replaces all occurrences of the pattern in the string

`split()`: splits the string by the occurrences of the pattern

`compile()`: compiles a regular expression pattern into a regular expression object, which can be used for efficient reuse

Expresiones regulares en Python

- Ejemplos:
 - Dado un texto obtener solo los números que aparezcan en él.

```
import re

sentence = """El Gobierno lanza el tercer intento para salvar Muface por 4.800 millones y da de plazo a las aseguradoras
             hasta el 4 de marzo."""

result = re.findall(r'\d+', sentence)

print(result)
```

```
['4', '800', '4']
```

Expresiones regulares en Python

- Ejemplos:
 - Dado un texto extraer las direcciones de email que aparezcan en él.

```
sentence = """Texto de prueba con alguna dirección de correo electrónico, la de la profesora (soto.montalvo@urjc.es),  
y la del coordinador del grado (grado.inteligenciaartificial@urjc.es)."""  
  
result = re.findall(r'[\w\.-]+@[\w]+[\.\w]*', sentence)  
  
print(result)  
  
['soto.montalvo@urjc.es', 'grado.inteligenciaartificial@urjc.es']
```

Referencias

- [1] Mansouri, B. Regular Expressions. Natural Language Processing. University of Southern Maine. 2023. ([Enlace](#))
- [2] Gerrero Carmona, F.M. y Marcos Aldón, M. Procesamiento del Lenguaje Natural. Neoinstrumenta. Revista del Laboratorio de Ingeniería Documental. 2014.
- [3] Documentación oficial de Python sobre regex. ([Enlace](#))
- [4] Plataforma para probar expresiones regulares. ([Enlace](#))